Back To Practice

Q1

Save

Completed

First-Come,First-Served CPU Scheduling Algorithm

Write a C program to implement the First-Come, First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

- **Completion Time (CT):** The time at which a process finishes execution.
- **Turnaround Time (TAT):** Total time taken from arrival to completion.
- $TAT = CT - AT$
- **Waiting Time (WT):** The time a process spends waiting in the ready queue.
- $WT = TAT - BT$

Input Format

1. The first input line contains an integer representing the number of processes.
2. The next lines contain two integers for each process, where the first integer represents the Arrival Time (AT) and the second integer represents the Burst Time (BT).

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Example 1

Sample Input 1

3

0 2

1 2

2 3

Sample Output 1

Enter number of processes:

Enter Arrival Time and Burst Time for P1:

Enter Arrival Time and Burst Time for P2:

Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33

Average Waiting Time: 1.00

Example 2

Sample Input 2

3

0 5

5 3

8 2

Sample Output 2

Enter number of processes:

Enter Arrival Time and Burst Time for P1:

Enter Arrival Time and Burst Time for P2:

Enter Arrival Time and Burst Time for P3:

Select Language : C (GCC 9.2.0)

```c
#include<stdio.h>
int main(){
  int n;
  printf("Enter number of processes: \n");
  scanf("%d",&n);
  int arr[n];
  int brr[n];
  for(int i=0;i<n;i++){
    printf("Enter Arrival Time and Burst Time for P%d: \n",i+1);
    scanf("%d",&arr[i]);
    scanf("%d",&brr[i]);

  }
  int comp[n];
  for(int i=0;i<n;i++){
    if(arr[i]==0&&i==0){
      comp[i]=brr[0];
    }
    else if(arr[i]<=comp[i-1]){
      comp[i]=comp[i-1]+brr[i];
    }
    else{
      comp[i]=comp[i-1]+(arr[i]-comp[i-1]+brr[i]);
    }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

> Test Case - 2